

Atividade 02 - Algoritmos de Busca (ESD)

Questão 1 - Criar vetores de diferentes tamanhos:

- 1.000 elementos
- 10.000 elementos
- 100.000 elementos

Vetores Criados:

```
Vetor<Integer> vetor1k = new Vetor<>(1000);  
Vetor<Integer> vetor10k = new Vetor<>(10000);  
Vetor<Integer> vetor100k = new Vetor<>(100000);
```

Questão 2 - Preencher os vetores com números inteiros aleatórios já ordenados

Preenchendo os vetores com valores aleatórios

```
// Preenchimento dos Vetores:  
for (int i = 0; i < 1000; i++) {  
    vetor1k.inserirFinal(random.nextInt());  
}  
vetor1k.ordena();  
  
for (int i = 0; i < 10000; i++) {  
    vetor10k.inserirFinal(random.nextInt());  
}  
vetor10k.ordena();  
  
for (int i = 0; i < 100000; i++) {  
    vetor100k.inserirFinal(random.nextInt());  
}  
vetor100k.ordena();
```

Questão 3 - Implementar duas funções de busca:

- buscaLinear(int[] vetor, int valor)
- buscaBinaria(int[] vetor, int valor)

```
public int buscaLinearOrdenada(T valor ) {
    zerarComparacoes();
    for (int i = 0; i < tamanho; i++) {
        if (elementos[i] != null) {
            if (elementos[i].equals(valor)) {
                return i;
            }

            // Faz uma ordenação simples, se o valor for
maior que o procurado,
            // já para de procurar na hora
            if (elementos[i].compareTo(valor) > 0) {
                return -1;
            }
        }
    }
    return -1;
}

//Possui complexidade O(logN), tornando assim mais rápido
na procura, pois divide o Array no meio
//Porém, o array PRECISA estar ordenado para funcionar
public int buscaBinaria(T valor) {
    zerarComparacoes();
    int inicio = 0;
    int fim = tamanho - 1;

    while (inicio <= fim) {
        int meio = inicio + (fim - inicio) / 2;

        if (elementos[meio] == null) {
```

```

        return -1; //Evita o NullPointerException caso
tenha buracos na lista
    }

    //Como o tipo T não aceita o operador < utilizamos
o compareTo para isso
    //Importante destacar que a classe precisa ser
extendida para o Comparable<T>

    int comparacao = elementos[meio].compareTo(valor);
    comparacoes++;

    //Procura no meio
    if (comparacao == 0) {
        return meio;
    }

    //Procura na direita
    if (comparacao < 0) {
        inicio = meio + 1;
    }

    //Procura na esquerda
    if (comparacao > 0) {
        fim = meio - 1;
    }
}
return -1; //Caso não encontre o elemento
}

```

Questão 4 - Para cada vetor, realizar buscas em três posições:

- Valor no início (primeiro elemento).
- Valor no meio (elemento central).
- Valor no fim (último elemento).

Com o método Auxiliar criado, facilita a busca e já calcula também o tempo:

```
//Método Auxiliar para calcular o tempo + comparações
private static void executarTesteBusca(String legenda,
Vetor<Integer> vetor, int indiceAlvo, boolean linear) {
    Integer valorProcurado = vetor.ler(indiceAlvo);

    long inicio = System.nanoTime();
    int resultadoIndice; //Indice informado

    if (linear) {
        resultadoIndice =
vetor.buscaLinearOrdenada(valorProcurado);
    } else {
        resultadoIndice =
vetor.buscaBinaria(valorProcurado);
    }

    long fim = System.nanoTime();
    long duracaoTotal = fim - inicio; //Conversão do tempo
total

    System.out.println "[" + legenda + "]";
    System.out.println "  -> Valor do Indice: " +
valorProcurado);
    System.out.println "  -> Indice encontrado: " +
resultadoIndice);
    System.out.println "  -> Comparacoes feitas: " +
Vetor.quantidadeComparacoes());
    System.out.println "  -> Tempo de execucao: " +
duracaoTotal + " ns (" + (duracaoTotal / 1000000.0) + "
ms)");
    System.out.println();
}
```

Implementação:

```
// ===== TESTES COM VETOR 1K =====

// --- BUSCA LINEAR ---
System.out.println("----- Vetor 1K
-----");
    executarTesteBusca("Linear - Inicio (Pos 0)", vetor1k,
0, true);
    executarTesteBusca("Linear - Meio (Pos 500)", vetor1k,
500, true);
    executarTesteBusca("Linear - Fim (Pos 999)", vetor1k,
999, true);

// --- BUSCA Binaria ---
    executarTesteBusca("Binaria - Inicio (Pos 0)", vetor1k,
0, false);
    executarTesteBusca("Binaria - Meio (Pos 500)", vetor1k,
500, false);
    executarTesteBusca("Binaria - Fim (Pos 999)", vetor1k,
999, false);

// ===== TESTES COM VETOR 10K
=====

// --- BUSCA LINEAR ---
System.out.println("----- Vetor 10K
-----");
    executarTesteBusca("Linear - Inicio (Pos 0)", vetor10k,
0, true);
    executarTesteBusca("Linear - Meio (Pos 5000)",
vetor10k, 5000, true);
    executarTesteBusca("Linear - Fim (Pos 9999)", vetor10k,
9999, true);

// --- BUSCA Binaria ---
    executarTesteBusca("Binaria - Inicio (Pos 0)",
vetor10k, 0, false);
    executarTesteBusca("Binaria - Meio (Pos 5000)",
vetor10k, 5000, false);
```

```

    executarTesteBusca("Binaria - Fim (Pos 9999)",
vetor10k, 9999, false);

    // ===== TESTES COM VETOR 100K
=====

    // --- BUSCA LINEAR ---
    System.out.println("----- Vetor 100K
-----");
    executarTesteBusca("Linear - Inicio (Pos 0)",
vetor100k, 0, true);
    executarTesteBusca("Linear - Meio (Pos 50000)",
vetor100k, 50000, true);
    executarTesteBusca("Linear - Fim (Pos 99999)",
vetor100k, 9999, true);

    // --- BUSCA Binaria ---
    executarTesteBusca("Binaria - Inicio (Pos 0)",
vetor100k, 0, false);
    executarTesteBusca("Binaria - Meio (Pos 50000)",
vetor100k, 50000, false);
    executarTesteBusca("Binaria - Fim (Pos 99999)",
vetor100k, 99999, false);
}

```

Questão 5 - Medir

- Comparações: adicionar um contador dentro das funções para contar quantas vezes uma comparação foi feita.
- Tempo de execução: usar a classe System.nanoTime() para capturar o tempo antes e depois da execução

```

//Método Auxiliar para calcular o tempo + comparações
private static void executarTesteBusca(String legenda,
Vetor<Integer> vetor, int indiceAlvo, boolean linear) {
    Integer valorProcurado = vetor.ler(indiceAlvo);

```

```

long inicio = System.nanoTime();
int resultadoIndice; //Indice informado

if (linear) {
    resultadoIndice =
vetor.buscaLinearOrdenada(valorProcurado);
} else {
    resultadoIndice =
vetor.buscaBinaria(valorProcurado);
}

long fim = System.nanoTime();
long duracaoTotal = fim - inicio; //Conversão do tempo
total

```

Com este método conseguimos fazer este cálculo

Analise Final

- Além da busca binária implementada manualmente, teste o método

Arrays.binarySearch:

```
import java.util.Arrays;
```

```
long inicio = System.nanoTime();
```

```
int pos = Arrays.binarySearch(vetor, valor);
```

```
long fim = System.nanoTime();
```

```
long duracaoMs = (fim - inicio) / 1000000;
```

```
System.out.println("Tempo Arrays.binarySearch: " + duracaoMs + " ms");
```

Avalie se há diferença significativa de tempo entre a implementação própria e a versão otimizada da biblioteca Java.

- Suponha que você esteja procurando uma palavra em um dicionário. O dicionário tem 240000 palavras.

Na pior das hipóteses, de quantas etapas você acha que a busca sequencial e busca binária precisaria?

Consegue generalizar para qualquer entrada n?